

Συστήματα Ανάκτησης Πληροφοριών (INF248)

Αναφορά Προγραμματιστικής Εργασίας – Φάση 2: Σημασιολογική ανάκτηση χρησιμοποιώντας Transformers και FAISS

Ρήγας Σάσσαλος

Αρ. Μητρώου: 3220178

Στη δεύτερη φάση της εργασίας, υλοποίησα ένα σύστημα Semantic Search. Σε αντίθεση με την Φάση 1 που βασίστηκε στην ακριβή ταύτιση λέξεων-κλειδιών (BM25), εδώ ο στόχος είναι η κατανόηση του νοήματος των κειμένων μέσω διανυσματικών αναπαραστάσεων.

Για την υλοποίηση χρησιμοποιήθηκαν:

- Η βιβλιοθήκη Sentence-Transformers για την μετατροπή των κειμένων σε διανύσματα (vectorization)
- Η βιβλιοθήκη FAISS (Facebook AI Similarity Search) για την αναζήτηση ομοιότητας
- Το trec_eval για την αξιολόγηση

Η διαδικασία υλοποιήθηκε με Python και χωρίστηκε σε βήματα για την βέλτιστη διαχείριση πόρων. Για την αναπαράσταση των εγγράφων και των ερωτημάτων, χρησιμοποίησα το μοντέλο all-MiniLM-L6-v2. Το συγκεκριμένο μοντέλο μετατρέπει κάθε κείμενο σε ένα διάνυσμα 384 διαστάσεων, τοποθετώντας κείμενα με παρόμοιο νόημα κοντά στον διανυσματικό χώρο.

Λόγω τεχνικών περιορισμών στο MacOS που εργαζόμουν και ασυμβατότητας μεταξύ των βιβλιοθηκών torch και faiss με conflicts σε thread του επεξεργαστή, η διαδικασία χωρίστηκε σε δύο στάδια:

- Αρχικά, παρήχθησαν τα embeddings για όλα τα έγγραφα (documents.csv) και τα ερωτήματα (queries.csv) και αποθηκεύτηκαν προσωρινά στον δίσκο
- Στην συνέχεια, έκανα επανεκκίνηση του kernel για την απελευθέρωση μνήμης και φόρτωσα την βιβλιοθήκη FAISS.

```
# Load Model & Generate Embeddings
model_name = 'all-MiniLM-L6-v2'
print(f"Loading Model {model_name}...")
model = SentenceTransformer(model_name)

print("Vectorizing Documents...")
doc_embeddings = model.encode(df_docs['Text'].tolist(), show_progress_bar=True)

print("Vectorizing Queries...")
query_embeddings = model.encode(df_queries['Text'].tolist(), show_progress_bar=True)

print("Saving embeddings to disk...")
np.save("doc_embeddings.npy", doc_embeddings)
np.save("query_embeddings.npy", query_embeddings)
np.save("doc_ids.npy", df_docs['ID'].values)
np.save("query_ids.npy", df_queries['ID'].values)
```

✓ 1m 56.4s

Python

```
Loading Model all-MiniLM-L6-v2...
Vectorizing Documents...
Batches: 100%|██████████| 573/573 [01:52<00:00, 5.08it/s]
Vectorizing Queries...
Batches: 100%|██████████| 1/1 [00:00<00:00, 9.31it/s]
Saving embeddings to disk...
```

Φόρτωσα τα αποθηκευμένα διανύσματα και τα μετέτρεψα σε μορφή float32 όπως είναι αναγκαίο για το FAISS. Εφάρμοσα L2 Normalization στα διανύσματα. Επίσης, χρησιμοποίησα το IndexFlatIP. Επειδή τα διανύσματα είναι κανονικοποιημένα, το εσωτερικό γινόμενο ισοδυναμεί μαθηματικά με την Cosine Similarity.

Για κάθε ερώτημα, ανέκτησα τα k = 50 πλησιέστερα έγγραφα.

```
# Build FAISS Index
print("Building FAISS Index...")
dimension = doc_embeddings.shape[1]
index = faiss.IndexFlatIP(dimension)
index.add(doc_embeddings)

print(f"Index ready with {index.ntotal} documents.")
✓ 0.0s Python
```

Building FAISS Index...
Index ready with 18316 documents.

```
# Save Results
results_dir = "results"
if not os.path.exists(results_dir):
    os.makedirs(results_dir)

output_file = f"{results_dir}/phase2_results.txt"
run_name = "Semantic_FAISS"

print(f"Writing results to {output_file}...")
with open(output_file, 'w') as f:
    for q_idx in range(len(query_ids)):
        q_id = query_ids[q_idx]

        for rank, doc_idx in enumerate(indices[q_idx]):
            real_doc_id = doc_ids[doc_idx]
            score = distances[q_idx][rank]

            f.write(f"{q_id} Q0 {real_doc_id} {rank+1} {score:.4f} {run_name}\n")

print(f"Done! Results saved to {output_file}")
✓ 0.0s Python
```

Writing results to results/phase2_results.txt...
Done! Results saved to results/phase2_results.txt

Τα αποτελέσματα αποθηκευτήκαν στο αρχείο phase2_results.txt και αξιολογήθηκαν με το trec_eval χρησιμοποιώντας το ίδιο αρχείο απαντήσεων (qrels.txt) με την Φάση 1.

```
IR_Phase2 — -zsh

Last login: Mon Jan 12 13:47:03 on ttys003
rigas@RigasMacbookAir IR_Phase2 ./trec_eval-9.0.7/trec_eval -m map -m P.5,10,15,|
20 data/qrels.txt results/phase2_results.txt
map          all      0.3741
P_5          all      0.6600
P_10         all      0.5000
P_15         all      0.3933
P_20         all      0.3500
rigas@RigasMacbookAir IR_Phase2
```

	Φάση 1 (BM25)	Φάση 2 (Semantic)	Διαφορά
MAP	0.7446	0.3741	-0.3705
P@5	0.8600	0.6600	-0.2000
P@10	0.7300	0.5000	-0.2300
P@15	0.6867	0.3933	-0.2934
P@20	0.6200	0.3500	-0.2700

Παρατηρούμε ότι η απόδοση του συστήματος της Φάσης 2 είναι χαμηλότερη σε σχέση με το μοντέλο BM25 της Φάσης 1. Αυτό οφείλεται σε διάφορους λόγους:

1. Πολλά ερωτήματα περιέχουν πολύ συγκεκριμένα ακρωνύμια έργων. Το BM25 είναι πολύ αποδοτικό στο να βρίσκει ακριβείς αντιστοιχίσεις σπάνιων όρων. Αντίθετα, το σημασιολογικό μοντέλο προσπαθεί να βρει κείμενα με παρόμοιο νόημα οπότε χάνει την ακριβή λέξη
2. Το all-MiniLM-L6-v2 έχει όριο εισόδου. Πολλά έγγραφα της συλλογής είναι μεγάλα, οπότε σημαντικές πληροφορίες που βρίσκονται στο τέλος του εγγράφου δεν λαμβάνονται υπόψη κατά το vectorization
3. Η σημασιολογική αναζήτηση βρίσκει σχετικά κείμενα που δεν έχουν τις ίδιες λέξεις αλλά για την συγκεκριμένη συλλογή που έχει πολύ εξειδικευμένη ορολογία η κλασική αναζήτηση λέξεων κλειδιών παραμένει πιο ακριβής.